DIGT2107: Practice of Software Development

Project Deliverable 3.2:

Implementation

Project Name:

SmartBudget – Personal Finance Tracker

Team Number:

Team #3

Team Members:

Mohsen Pasdar, Muhammed Ahmed, Alia Hagi-Dhaffe

Date:

March 08, 2026

Table of Contents

1 Introduction 1

2 Starting Point from Deliverable 3.1 2

3 Refactoring and Codebase Improvements 3

3.1 Refactoring Goals 3

3.2 Codebase Improvements Applied in Iteration 3.2 3

3.2.1 Minor cleanup for clarity and consistency 3

3.2.2 Small reduction of duplicated validation logic 4

3.2.3 Keeping persistence responsibilities separated 4

3.2.4 Minor updates to support testing and validation 4

3.3 Impact of Improvements 4

3.3.1 Improved readability and maintainability 4

3.3.2 Better consistency across workflows 4

3.3.3 Stronger support for testing and validation 4

4 Final Implementation of Core Features 5

4.1 Core functionality already completed in Deliverable 3.1 5

4.2 Key core features completed in Deliverable 3.2 5

4.3 Integration and consistency across the system 6

5 Testing and Validation 6

5.1 Testing Strategy and Approach 7

5.2 Functional Testing 7

5.2.1 Transactions (Record/Edit/Delete) 7

5.2.2 View & Filter Transactions 8

5.2.3 Budgets and Overspending Alerts 8

5.2.4 Reports (Summary + Category Breakdown) 9

5.2.5 Persistence behaviors (Save/Load) 9

5.3 System and Integration Testing 10

5.4 Edge Cases and Error Handling 11
5.5 Defects Found, Fixes, and Retesting 11
6 Updated Backlog 12
6.1 Completed Backlog Items 12
6.2 Remaining Backlog Items (Next Iteration) 13
7 Code Repository 13
7.1 Repository Link: 13
7.2 Submission Tag 14
8 Conclusion 14

1 Introduction

SmartBudget is a personal finance application that helps users record daily transactions, set monthly budgets by category, and view summary reports to better understand their spending habits. The main goal of the system is to make personal budgeting simple and practical by giving users a clear view of their income and expenses, while also helping them manage spending through budget tracking, alerts, and summary reports. Across the earlier deliverables, our team developed the project step by step, beginning with requirements, use cases, and testing documentation, and then moving into software design, GUI development, and the initial implementation of the system.

This document represents Deliverable 3.2 for the SmartBudget project. The main purpose of this iteration is to complete and strengthen the implementation by refactoring the existing codebase, finishing important backlog items from Deliverable 3.1, and carrying out more complete testing and validation. In particular, this report focuses on codebase improvements for better structure and maintainability, the addition of remaining features such as persistence and loading saved data on startup, and the testing work needed to confirm that the system functions correctly after integration. These areas match the main requirements of the Deliverable 3.2, which emphasizes refactoring, final implementation of core features, and full testing and validation.

In the previous deliverable, the system already supported the main workflows during an active session, including transaction management, budgeting, report generation, and basic GUI interactions. At the same time, some work was left for the next cycle, especially persistence and additional test coverage for alert and reporting logic. For that reason, this deliverable builds directly on Deliverable 3.1 and shows the progress made in turning the project into a more complete, integrated, and reliable application.

This report is organized to reflect the main goals of the iteration. It begins with the refactoring and codebase improvements completed by the team, followed by the final implementation of the remaining core features, and then presents the testing and validation carried out to verify correctness, integration, and system stability. Together, these sections show the progress made in this stage of the project and prepare the system for the final review and submission.

2 Starting Point from Deliverable 3.1

Deliverable 3.2 builds directly on the work completed in Deliverable 3.1, where the SmartBudget project moved from earlier planning and prototype work into a functional implementation stage. In Deliverable 3.1, the team completed the main software design documentation, including the layered architecture overview, UML class diagrams, sequence diagrams, and activity diagrams for the system's key workflows. At the same time, the project reached an important development milestone because the main modules of the application were already implemented and connected to the JavaFX interface. These modules included transaction management, transaction viewing and filtering, monthly budgets, overspending alerts, summary reports, and navigation across the main pages of the system.

By the end of Deliverable 3.1, SmartBudget was able to demonstrate the main user workflows during an active session and already reflected the core requirements defined earlier in Deliverable 2.2. This showed that the project had moved beyond the design and prototype stage and into a working desktop application with a clear layered structure separating the UI, business logic, domain model, and persistence concerns. In other words, the starting point for Deliverable 3.2 was not an incomplete design, but a working system that already supported the main features at a functional level.

At the same time, Deliverable 3.1 clearly identified the main work that still remained for the next cycle. The most important limitation was that data persistence had not yet been finalized, which meant that transactions and budgets were not saved after the application closed. The preliminary testing and coverage review also showed that some important areas still needed stronger automated testing, especially alert logic, report generation, and the persistence-related workflows planned for the next stage. In addition, a few small GUI refinements were still listed in the backlog.

For this reason, Deliverable 3.2 begins from a project state that is already stable in terms of design and core functionality, but still needs completion in terms of persistence, broader testing, and final code quality improvements. The updated backlog in Deliverable 3.1 also confirms this direction by showing that most major design and implementation stories were already done, while the remaining work focused mainly on persistence, missing test coverage, and a few usability improvements. Therefore, the purpose of this iteration is to build on the existing implementation, improve its internal quality through refactoring, complete the unfinished backlog items, and validate the system more thoroughly as a complete and integrated application.

3 Refactoring and Codebase Improvements

In Deliverable 3.2, we focused on improving the overall quality and maintainability of the SmartBudget codebase while completing the remaining core implementation work. Since the main workflows and system structure were already working well after Deliverable 3.1, this iteration did not involve large-scale restructuring. Instead, we made a set of small, practical improvements as part of the persistence implementation and final integration work. These updates helped keep the code clean, consistent, and easier to test as the system moved closer to completion, which aligns with the Deliverable 3.2 expectations around codebase improvement and validation.

3.1 Refactoring Goals

The goal of the refactoring work in this iteration was mainly to support a smoother final implementation and testing phase. The improvements were focused on keeping the existing design consistent and avoiding unnecessary complexity while adding persistence and expanding validation.

- Improve readability and consistency in the implementation
- Reduce small cases of repeated logic (especially input validation and common checks)
- Keep responsibilities clear between UI, services, domain models, and persistence
- Support testing by keeping business logic organized and easier to validate
- Ensure persistence integration fits cleanly into the existing structure

Overall, these changes helped reduce the chance of integration issues and made the system easier to maintain during the final stage of development.

3.2 Codebase Improvements Applied in Iteration 3.2

The improvements in this iteration were mostly incremental and tied to integrating persistence and ensuring the application remains stable after saving and loading data.

3.2.1 Minor cleanup for clarity and consistency

Some service methods and UI handlers were adjusted for better clarity and consistency. This included small formatting and naming improvements, as well as minor restructuring to keep methods focused and easier to follow.

3.2.2 Small reduction of duplicated validation logic

In a few places, similar validation checks were repeated across workflows. These checks were made more consistent by consolidating simple validation steps where appropriate, which helps ensure the same rules are applied across the application.

3.2.3 Keeping persistence responsibilities separated

As persistence and startup loading were implemented, care was taken to keep save/load logic separate from unrelated UI or business logic. This helped preserve the intended layered structure and made it easier to test persistence without affecting other modules.

3.2.4 Minor updates to support testing and validation

Some small structural adjustments were made to support testing and validation, especially around persistence-related workflows. Where needed, methods were kept smaller and clearer so that behavior could be validated more directly through unit tests and integration checks.

3.3 Impact of Improvements

Although the refactoring in this deliverable was not extensive, the improvements still supported the overall quality of the final system.

3.3.1 Improved readability and maintainability

The code remains easier to understand and maintain, particularly in areas affected by persistence and integration.

3.3.2 Better consistency across workflows

By keeping validation and responsibility boundaries consistent, the system is less likely to behave differently across screens or features, which reduces confusion and prevents small logic mistakes.

3.3.3 Stronger support for testing and validation

These improvements helped support broader testing and validation in Deliverable 3.2 by keeping logic cleaner and more predictable, which is important for confirming the system works correctly after integration.

4 Final Implementation of Core Features

Deliverable 3.2 focuses on completing SmartBudget as a fully functional and integrated application. Since the core features and main workflows were already implemented in Deliverable 3.1, this iteration did not require building the system from scratch. Instead, the goal was to finalize the remaining essential features from the project backlog and ensure that the implementation matches the requirements and design that were documented earlier. This aligns directly with the Deliverable 3.2, which emphasizes feature completeness, integration across modules, and a system that is ready for final testing.

4.1 Core functionality already completed in Deliverable 3.1

By the end of Deliverable 3.1, SmartBudget already supported the main user workflows through the JavaFX interface. The system included working modules for recording, editing, deleting, and filtering transactions, managing monthly budgets, triggering overspending alerts, generating summary reports, and navigating between the Transactions, Budgets, and Reports pages. The project also included JavaFX modals and confirmations to make user actions clearer and reduce mistakes, along with initial unit testing and manual GUI smoke testing for the main workflows.

Even though the system was functional during an active session, Deliverable 3.1 clearly documented one major limitation: data persistence was not yet implemented, meaning that transactions and budgets would not be saved after the application closed. The report also stated that this would be addressed in the next deliverable.

4.2 Key core features completed in Deliverable 3.2

The main implementation focus in Deliverable 3.2 was to finish the remaining “must-have” items from the project backlog that were required to make SmartBudget usable across multiple sessions. Based on the backlog status from Deliverable 3.1, the most important items planned for Iteration 3.2 were:

- implementing the persistence layer (Storage abstraction + JSON save/load integration),
- loading saved data automatically on startup,
- adding tests to confirm persistence behavior stays correct and does not break existing workflows.

These changes complete an essential part of the system's original functional requirements, since persistence was always expected to be part of SmartBudget so that users can keep their transaction and budget history over time.

In addition to persistence and startup loading, Deliverable 3.2 also aims to finalize any incomplete functionality from the previous iteration and ensure that all modules work together consistently. This includes confirming that the Transactions, Budgets/Alerts, and Reports modules interact correctly after persistence is added, and that the GUI continues to behave correctly with saved and restored data. This is important because adding persistence affects the full data flow of the application and can expose integration issues if modules do not handle saved state properly.

4.3 Integration and consistency across the system

After implementing persistence and startup loading, the system is treated as a single integrated application rather than separate features that only work in isolation. For Deliverable 3.2, the emphasis is on ensuring that the entire workflow remains correct end-to-end. For example, once data is loaded at startup, users should be able to view existing transactions, filter them, edit or delete them, generate summary reports for a chosen date range, and still have budgets and overspending alerts behave correctly based on the restored data. This supports the deliverable requirement that all features are fully developed, integrated, and ready for final testing and validation.

5 Testing and Validation

The goal of testing in Deliverable 3.2 is to confirm that SmartBudget is stable, correct, and ready for final review as a complete system. In this iteration, testing focuses on two major areas. First, we validate that each core feature meets its functional requirements through comprehensive functional testing, including edge cases and error conditions. Second, we perform system and integration testing to confirm that the full workflow works end-to-end and

that modules interact correctly, especially after persistence and startup loading were implemented.

5.1 Testing Strategy and Approach

Our testing strategy combines automated unit tests with functional workflow testing. Automated tests are used to validate business rules in the domain/service layers (independent from the GUI), while functional and integration testing validate that real user workflows behave correctly through the application interface. This approach builds on our earlier testing direction from Deliverable 3.1, where we used JUnit unit tests for core logic and manual GUI smoke tests for workflow validation, and also addresses the coverage gaps identified for alerts, reporting, and persistence.

5.2 Functional Testing

Functional testing verifies that each major feature behaves correctly and meets its requirements, including normal scenarios, error handling, and edge cases. Test cases were designed to cover the main functions of SmartBudget (transactions, filters, budgets, alerts, reports, and persistence) and validate the expected behavior when users provide both valid and invalid inputs.

5.2.1 Transactions (Record/Edit/Delete)

Valid transaction creation (income/expense), handling invalid inputs such as negative amounts, non-numeric amounts, missing required fields, and invalid dates were tested.
Editing transactions was performed to confirm the list and totals updated accordingly.
Deleting transactions was tested with confirmation prompts and cancel-delete behaviors to ensure proper user control.

Table 5-1: Tests results - Transactions

Test Description	Status
Successfully create a valid transaction for both income and expense entries	Pass
Reject invalid inputs including negative amounts, non-numeric values, missing required fields, and invalid dates	Pass
Edit an existing transaction and verify that the transaction list and totals update accordingly	Pass
Delete a transaction with user confirmation and verify that cancelling the deletion retains the transaction	Pass

5.2.2 View & Filter Transactions

Filtering transactions by date range and category was tested, along with combined filters. The system's response to empty result sets, showing "no records found" messages, was also validated.

Table 5-2: Tests results – View and Filter Transactions

Test Description	Status
Filter transactions by a specified date range and verify only matching transactions are shown	Pass
Filter transactions by category and verify correct transactions are displayed	Pass
Apply multiple filters together and confirm the filtered results are correct	Pass
Handle empty result sets correctly, showing appropriate "No transactions yet" message	Pass

5.2.3 Budgets and Overspending Alerts

Creating, editing, and deleting budgets were tested for correctness. Overspending detection was verified by confirming alerts triggered when spending reaches or exceeds set budget limits.

Table 5-3: Tests results - Budgets and Overspending Alerts

Test Description	Status
Create a new budget and verify it is saved and displayed correctly	Pass
Edit an existing budget and confirm the updated value is saved correctly	Pass
Delete a budget and confirm it is removed successfully	Pass
Trigger an overspending alert when total expenses exceed the defined budget and verify alert displays	Pass

5.2.4 Reports (Summary + Category Breakdown)

Generating financial reports for valid periods was tested, including verification of correct date range handling, especially when start dates are after end dates. Reports for empty periods (no transactions) were tested to ensure they display zero values. Calculations of income totals, expense totals, balances, and category breakdowns were validated.

Table 5-4: Tests results - Reports

Test Description	Status
Generate a report for a valid period and verify totals for income, expenses, and balance	Pass
Test report generation with date range boundaries and ensure correct ordering	Pass
Generate a report for a period with no transactions and verify zero values are displayed	Pass
Verify all calculations including category totals, income total, expense total, and overall balance	Pass

5.2.5 Persistence behaviors (Save/Load)

Because persistence is a key completion goal for Iteration 3.2, functional testing includes verifying that transactions and budgets are stored correctly and restored accurately across sessions. This directly supports FR1 (persist across sessions) and the “load saved data on startup” user story expectations documented earlier.

Table 5-5: Tests results - Persistence

Test Description	Status
Save data after adding, editing, or deleting a transaction and confirm persistence	Pass
Save data after adding, editing, or deleting a budget and confirm persistence	Pass
Restart the application and confirm all saved data (transactions and budgets) are restored correctly	Pass

5.3 System and Integration Testing

System and integration testing validates that the full application works end-to-end and that features interact correctly, especially after persistence and refactoring changes. These tests focus on workflow completion, data consistency, and communication between modules (Transactions ↔ Budgets/Alerts ↔ Reports) as required by the 3.2 Deliverable.

Table 5-6: Tests results - System and Integration

Workflow ID	Scenario	Expected Result	Actual Result	Status	Notes
WF-001	Add transactions, set a budget, and trigger overspending alert	Overspending alert displays when spending exceeds budget	Overspending alert displayed correctly	Pass	N/A
WF-002	Add transactions, generate report, and verify totals and category breakdown	Report totals and category breakdown are accurate	Report generated correctly	Pass	N/A
WF-003	Save data, restart the app, load data, and generate report from restored data	Report generated correctly from restored transactions and budgets	Report generated correctly	Pass	N/A
WF-004	Save data, restart the app, edit or delete a loaded transaction, and confirm persistence updates	Edited/deleted transaction updates are saved and reflected	Changes persisted correctly	Pass	N/A
WF-005	Save data, restart the app, and confirm budgets and alerts still function with loaded transactions	Budgets and alerts function correctly after restart	Budgets and alerts worked correctly	Pass	N/A

5.4 Edge Cases and Error Handling

To ensure robustness, we tested edge cases and error conditions across core features. This includes invalid user inputs, empty datasets, boundary dates, and unusual sequences of actions (e.g., deleting items used in reports, editing items after loading saved data). The purpose is to verify the system behaves predictably and provides clear user feedback rather than crashing or producing incorrect results.

No critical defects were found during this iteration. Minor UI/validation adjustments were made during development and verified through regression testing.

Table 5-7: Tests results - Edge Cases and Error Handling

Edge Case ID	Scenario	Expected Result	Actual Result	Status
EC-001	Enter invalid transaction amounts, including negative numbers or non-numeric values	Transaction rejected with an error message explaining the input is invalid	Error message displayed	Pass
EC-002	Generate a report using a date range where the start date is after the end date	Validation message displayed indicating the date range is incorrect	Correct validation message shown	Pass
EC-003	Generate a report for a period that has no transactions recorded	Report shows zero values for income, expenses, and balance	Report displayed correctly	Pass

5.5 Defects Found, Fixes, and Retesting

During testing, any discovered defects were carefully documented, including a description of the issue, steps to reproduce it, and its impact on system functionality. After each fix, the affected test cases were re-executed to confirm that the defect was resolved and that the system continued to function correctly. This iterative process ensured that all critical and high-priority issues were resolved before final submission, maintaining system stability and reliability. Additionally, regression testing was performed to verify that previously tested features continued to operate as expected following each change.

6 Updated Backlog

This section summarizes the backlog status for Iteration 3.2 and shows the progress made since the previous deliverable. The backlog items are based on the design stories introduced in Deliverable 3.1 and track the remaining development tasks required to reach a fully functional system.

During this iteration, the focus of development was on completing the persistence functionality and integrating it with the existing modules of the system. This work allows the application to store and restore user data across sessions, which was identified as the main missing capability in the previous iteration.

6.1 Completed Backlog Items

The following backlog items were completed during this iteration:

Table 6-1: Updated Backlog - Completed Items

ID	Design Story (Developer Story Format)	Related User Story	Priority	Dependency	Status	Target
DS-11	As a developer, I have implemented a persistence layer (Storage interface + JSON save/load + integration) so that application data is saved and restored across sessions.	US-8	High	DS-01	Done	Iteration 3.2
DS-12	As a developer, I have implemented loading saved data on startup so that users automatically see their saved transactions and budgets when launching the application.	US-8	High	DS-11	Done	Iteration 3.2
DS-13	As a developer, I have written unit tests for persistence (save/load correctness + restart regression tests) so that persistence behavior is reliable and does not break existing features.	US-8	High	DS-11, DS-12	Done	Iteration 3.2

These completed items significantly improve the reliability of the SmartBudget system by ensuring that user data is not lost when the application closes. The persistence layer now integrates with the service layer and domain objects, allowing transactions and budgets to be stored and restored correctly.

6.2 Remaining Backlog Items (Next Iteration)

The remaining backlog items will be addressed in the next development cycle. These tasks focus mainly on improving testing coverage and refining specific parts of the system.

Table 6-2: Updated Backlog - Items To Do Next Report

ID	Design Story (Developer Story Format)	Related User Story	Priority	Dependency	Status	Target
DS-14	As a developer, I have added unit tests for AlertService so that overspending alert rules (including boundary cases) are verified and UC-6 is validated.	US-5	Medium	DS-05	To Do	Iteration 3.3
DS-15	As a developer, I have added unit tests for ReportService and PeriodRange so that report calculations, date range validation, and empty-result behavior are correct and UC-5 is validated.	US-6	Medium	DS-06	To Do	Iteration 3.3
DS-17	As a developer, I have improved the transaction form UI to filter categories by transaction type (Income vs Expense) so that users only see relevant categories and input errors are reduced.	US-1	Medium	DS-08	To Do	Iteration 3.3

At the end of Iteration 3.2, the SmartBudget application now supports all core functional workflows defined in the project requirements, including transaction management, budgeting, reporting, and persistent storage of user data.

The remaining work primarily focuses on expanding automated testing and refining parts of the user interface to improve usability and maintainability. These improvements will be addressed in the next iteration as part of the final system refinement phase.

7 Code Repository

The SmartBudget project is hosted on GitHub, and all project deliverables, documents, and source files are stored and updated there throughout each iteration.

7.1 Repository Link:

The full project repository can be accessed at:

https://github.com/Hercules-Hawk/smart-budget-personal-finance-tracker

All project updates, including the final implementation, refactored code, and updated tests for this iteration, have been pushed to this repository.

7.2 Submission Tag

To mark the submission for this deliverable, the current version of the repository has been tagged as: **ITR3.2**

This tag identifies the exact commit corresponding to the Iteration 3.2 submission and allows the instructor and TA to review the specific version of the codebase associated with this deliverable.

8 Conclusion

In this deliverable, the SmartBudget project was advanced from an initial functional prototype into a more complete and stable implementation. Building on the design and GUI work completed in Deliverable 3.1, this iteration focused on strengthening the system through refactoring, completing the remaining core functionality, and improving the overall reliability of the application. The project now supports the main workflows of recording transactions, managing budgets, generating reports, and preserving data across sessions through the newly completed persistence layer.

A major part of this iteration was the implementation of persistence and startup loading, which addressed the main limitation identified in the previous deliverable, where application data was not saved after the program closed. With this improvement, SmartBudget now provides a more complete user experience, since transactions and budgets can be stored and restored correctly. In addition, the codebase was further refined to support maintainability and consistency with the layered architecture defined earlier, keeping the separation between UI, service, domain, and persistence responsibilities clear.

This deliverable also included continued testing and validation of the implemented system. Along with the previously established GUI workflows and service-layer functionality, the completed implementation for this iteration improves confidence that the system behaves correctly as an integrated application. The backlog status for Iteration 3.2 shows that the persistence-related stories were completed successfully, while the remaining work has now been narrowed mainly to additional testing coverage and a few interface refinements.

Overall, SmartBudget has now reached a strong stage of development and reflects steady progress across all project iterations. The system is functionally much closer to the final expected version, and the remaining work is focused more on refinement than on missing core functionality. This provides a solid foundation for the next deliverable, where the team can

concentrate on final improvements, broader validation, and preparing the project for final submission.